Paper code: CSC-501 / CEN-504 Old

Roll No...23BCS066.....

B. Tech. (Computer Engineering) V Semester Examination, 2025-26

Time: 3 Hours **Object Oriented Programming (Paper code: CSC-501 / CEN-504 Old)** **Max. Marks: 45/60**

Note: Attempt any two parts from each question.
Make suitable assumptions wherever necessary.

Q1. **(a)** How do private, public, and protected access modifiers control data hiding and accessibility in Object-Oriented Programming? Explain their differences with an example, demonstrating access from within and outside a class in C++. **[4.5/6] [BL4, 5]**

CO1

(b) Design a C++ program for a Restaurant by defining a class named *Order* with following *data members*: order id, table number, food items ordered, total payable amount, the current status of the order (Pending / Preparing / Served / Billed / Cancelled). **[4.5/6] [BL6]**

Member functions:

placeOrder () - input and initialize all details (default status = "Pending").

updateStatus () - modify status to one of the allowed values.

generateBill () - display complete order details.

Implement a menu-driven interface that allows the user to invoke any of the member functions.

(c) Explain the following C++ programming constructs with examples codes highlighting their functionality **[4.5/6] [BL3,4]**

(i) Inline function

(ii) Dynamic Constructor

(iii) Return by reference

Q2. **(a)** Implement a class *Date* that contains three integer data members to represent day, month and year. Write the necessary code such that the following driver function runs correctly and performs the desired operations. **[4.5/6] [BL3]**

CO2

```
int main()
{
        Date D1(28, 2, 2024);
        Date D2(31, 12, 2023);
        cout << "Original D1: " << D1 << endl;
        cout << "Original D2: " << D2 << endl;
        ++D1;
        ++D2;
        cout << "After Increment D1: " << D1 << endl;
        cout << "After Increment D2: " << D2 << endl;
        return 0;
};
```

(b) Explain the situations in which a friend function is required to pass arguments by referen and those in which passing by value is more appropriate. Design two classes *Account* and *Loan* to represent a simple bank transaction system. Each account has a *balance*, and each loan has an *amount* to be deducted. Create a friend function *deductLoan(...)* that directly deducts the loan amount from the account balance.

(c) Implement all functions for the following class in C++ to perform Matrix operations. [4.5/6] [BL3]

```
#include <iostream>
using namespace std;
class Matrix
{
        int mat[10][10];
        int rows, cols;
    public:
        Matrix();                          // Default constructor
        Matrix(int r, int c);              // Parameterized constructor
        void input();                      // Input matrix elements
        void display();                    // Display matrix
        Matrix operator*(Matrix &);        // Matrix multiplication
        Matrix operator~();                // Transpose
        bool operator==(Matrix &);         // Check equality
};
```

Q3. (a) Demonstrate the concept of virtual base class with the help of a suitable program. [4.5/6] [BL3,4]

CO3

(b) Design a C++ program using multilevel inheritance to manage hotel guest information. In a hotel, some customers may stay for short period, while others opt for long-stay contracts (have some discounts), The base class *Guest* includes the data members *name* and *phone*, with member functions to *input* and *display* guest details. The derived class *Boarder*, which inherits from *Guest*, adds data members *roomNo* and *tariff*, along with functions to *input* and *display* room details. Further, the derived class *LongStay*, which inherits from *Boarder*, introduces two additional data members *contractNo* and *discountPercent* and provides functions to *input* and *display* contract details. The program must implement both default and parameterized constructors with proper constructor chaining. [4.5] [BL3,6]

(c) Differentiate between class templates and function templates. Design a template-based C++ function to sort arrays of any data type. The function should work for multiple types such as int, float, and char, illustrating the concept of generic programming. [4.5/6] [BL4,6]

(b) Discuss the significance of the *super* keyword in Java briefly. Illustrate its usage in calling parent class constructors and methods. [4.5/6] [BL2,3]

(b) Differentiate between abstract class, interface, and concrete class with example code snippets in Java. [4.5/6] [BL4]

(c) Design a Java program to manage premium subscriber information for an online streaming platform. The system should use inheritance to represent common and specialized subscriber details. The base class, *Subscriber*, includes data members *subscriberId* (integer) and *name* (String) to store basic subscriber information, along with a static data member *platformName* (String). It should provide a constructor to initialize the subscriber's ID and name, and a static method *setPlatformName(String name)* to assign the platform name for all subscribers. [4.5/6] [BL6]

A derived class, *PremiumSubscriber*, inherits *Subscriber* and adds two additional data members *plan* (String) and *monthlyFee* (double) to store the subscription plan and monthly charges. The subclass should include a constructor to initialize all attributes, including those inherited from Subscriber, and a method to display details of subscribers whose monthlyFee exceeds Rs. 799.

In the main () method, the program should first set the platform name using the static method, then create an array of PremiumSubscriber objects initialized with sample data. Finally, it should display the details of those subscribers whose monthly subscription fee is greater than Rs. 799.

Q5. (a) Define Generics in Java and explain how they improve type safety compared with using Object references. Also, explain the purpose of the following line in a program: [4.5/6] [BL1,2]

CO5

public static <T extends Comparable<T>> void sort_generics(T[] a)

(b) In an online shopping application, the user is required to enter the quantity of items they wish to purchase. Sometimes, users may mistakenly enter invalid inputs such as alphabets, symbols, or leave the field empty, causing the program to crash. To handle this situation, design a Java program that validates user input using exception handling. The program should prompt the user to enter the quantity and use a *try-catch-finally* block to handle errors. It should catch both *InputMismatchException* and *NumberFormatException* if the entered quantity is not a valid number. When an invalid input is entered, the program should display an appropriate error message like "Invalid quantity! Please enter digits only." If the input is valid, it should display the quantity entered by the user and complete execution gracefully. [4.5/6] [BL6]

(c) Design a Java program to simulate a simple bank account system using custom exception handling. Define a class *BankAccount* with a method *withdraw(double amount)* that throws a user-defined exception *InsufficientBalanceException* when the withdrawal amount exceeds the available balance. In the main() method, use a *try-catch* block to handle the exception and display suitable messages for both successful and unsuccessful withdrawal attempts. [4.5/6] [BL6]